

Improving Automatic Parallel Training via Balanced Memory Workload Optimization

Yujie Wang , Youhe Jiang , Xupeng Miao , Fangcheng Fu , Shenhan Zhu , Xiaonan Nie , Yaofeng Tu ,
and Bin Cui , *Fellow, IEEE*

Abstract—Transformer models have emerged as the leading approach for achieving state-of-the-art performance across various application domains, serving as the foundation for advanced large-scale deep learning (DL) models. However, efficiently training these models across multiple GPUs remains a complex challenge due to the abundance of parallelism options. Existing DL systems either require manual efforts to design distributed training plans or limit parallelism combinations to a constrained search space. In this paper, we present Galvatron-BMW, a novel system framework that integrates multiple prevalent parallelism dimensions and automatically identifies the most efficient hybrid parallelism strategy. To effectively navigate this vast search space, we employ a decision tree approach for decomposition and pruning based on intuitive insights. We further utilize a dynamic programming search algorithm to derive the optimal plan. Moreover, to improve resource utilization and enhance system efficiency, we propose a bi-objective optimization workflow that focuses on workload balance. Our evaluations on different Transformer models demonstrate the capabilities of Galvatron-BMW in automating distributed training under varying GPU memory constraints. Across all tested scenarios, Galvatron-BMW consistently achieves superior system throughput, surpassing previous approaches that rely on limited parallelism strategies.

Index Terms—Transformers, distributed learning, automatic parallelism.

I. INTRODUCTION

TRANSFORMER models have achieved great success in a wide range of deep learning (DL) applications in recent years, such as computer vision (CV) [1], [2], natural language

processing (NLP) [3], [4], [5], [6], [7], graph learning [8], [9] and recommendation systems [10]. For example, many Transformer variants (e.g., BERT [11], GPT-2 [12], T5 [13]) are leading the state-of-the-art performance in various NLP tasks such as machine translation and question answering. Transformers are also applicable to image recognition (e.g., ViT [1], Swin Transformer [14]) and multimodal tasks (e.g., CLIP [15], DALL-E [16]). Due to their superior performance, Transformers are becoming increasingly important in modern artificial intelligence industries.

Empirical evidence indicates that scaling model parameters is an effective path towards model performance improvements [17]. For instance, the original Transformer only has millions of model parameters while GPT-2 has 1.5 billion with superior performance [12]. Such large amounts of parameters also incur high computational and memory costs even for emerging accelerators like GPUs. With the increasing model scales, building and designing Transformers demand more system optimizations, and *how to perform efficient Transformers training* is becoming more challenging.

Distributed DL systems adopt data and model parallelism to improve the training efficiency by utilizing multiple GPU devices. Data parallelism divides the large volume of input data into multiple parts and each device is only responsible for partial data [18], [19], [20]. It requires each device to store a whole model replica, suffering from large model scales. Model parallelism is a more promising direction that partitions the model from different *parallelism dimensions* and makes each device store a subset of model parameters, such as tensor parallel [21] and pipeline parallel [22], [23], [24], [25]. Various choices of the parallelism strategies lead to distinct memory consumption, communication overheads, and execution efficiency.

However, directly applying these techniques to scaling Transformers is facing crucial challenges in both system efficiency and usability. Some recent advanced methods have been proposed to automatically find the parallelism strategies through the fine-grained combination of data and model parallelism for individual operators in the model. For example, OptCNN [26], FlexFlow [27], [28], Tofu [29], and TensorOpt [30] consider both data and tensor parallelism and use different search algorithms to optimize the execution plans. PipeDream [24] and DAPPLE [31] combine pipeline parallelism with data parallelism to improve the efficiency. Unfortunately, existing approaches only support limited parallelism dimensions (i.e., data parallelism and rare model parallelism dimensions) or rely on strong model and

Manuscript received 3 July 2023; revised 1 February 2024; accepted 13 February 2024. Date of publication 27 February 2024; date of current version 12 July 2024. This work was supported in part by the National Key R&D Program of China under Grant 2022ZD0116315, in part by NSFC under Grant U23B2048 and under Grant U22B2037, in part by China National Postdoctoral Program for Innovative Talents under Grant BX20230012, in part by Beijing R&D Program under Grant Z231100010323002, in part by Beijing NSF under Grant 4244080, and in part by ZTE-PKU joint program. Recommended for acceptance by F. Rusu. (Corresponding author: Yujie Wang.)

Yujie Wang, Youhe Jiang, Fangcheng Fu, Shenhan Zhu, and Xiaonan Nie are with the School of CS and Key Lab of High Confidence Software Technologies (MOE), Peking University, Beijing 100871, China (e-mail: alfredwang@pku.edu.cn; youhejiang@gmail.com; ccchengff@pku.edu.cn; shenhan.zhu@pku.edu.cn; xiaonan.nie@pku.edu.cn).

Xupeng Miao is with Computer Science Department, Carnegie Mellon University, Pittsburgh, PA 15213 USA (e-mail: xupeng@cmu.edu).

Yaofeng Tu is with the ZTE Company, Beijing 214135, China (e-mail: tu.yaofeng@zte.com.cn).

Bin Cui is with the School of CS and Key Lab of High Confidence Software Technologies (MOE), Peking University, Beijing 100871, China, and also with the Institute of Computational Social Science, Peking University, Qingdao 266102, China (e-mail: bin.cui@pku.edu.cn).

Digital Object Identifier 10.1109/TKDE.2024.3370614

hardware configurations (i.e., expert-designed parallelism strategy) and result in sub-optimal performance in practice. To the best of our knowledge, there are few prior works considering the automatic parallelism for large-scale Transformers with a complex search space including multiple parallelism dimensions.

In this approach, we propose Galvatron-BMW, a novel automatic parallel training system for Transformer models over multiple GPUs. Our target is to integrate data parallelism with a variety of model parallelism dimensions, provide a rarely larger search space (compared with previous approaches), and find the optimal hybrid parallelism strategies in an efficient manner. However, such an integration brings an explosive growth of the search space and cannot be directly explored as usual. Therefore, we are interested in the following question: *How can we exploit as many parallelism dimensions as possible and efficiently explore the search space in the meanwhile?*

We study five parallelism paradigms, four of which are popular parallelism paradigms in the distributed training of Transformer models, including data parallelism (DP), sharded data parallelism (SDP) [32], tensor parallelism (TP), and pipeline parallelism (PP). Besides, we also take into account activation checkpointing (CKPT) as a special parallelism dimension, which distributes the training memory workload to the backward computation through checkpoints. These parallelism paradigms have distinct memory consumption and communication overheads and no single paradigm could beat the others on both sides. The search space of automatic parallelism should include the arbitrary combinations of them. Inspired by some key intuitions from our observations and analysis, we first propose a decision-tree structure to decompose the search space and perform pruning to remove the inefficient combinations. To determine the final distributed execution plan, we then propose a dynamic programming search algorithm to utilize the optimal substructure property of this problem. Based on these, we provide **Galvatron-BMW**, which not only targets automatic parallelism for Transformer model training, but also considers the **B**alancing trade-off between **M**emory and computation **W**orkloads across devices. During the search process, Galvatron-BMW provides the required computation and communication costs and memory consumption through a cost estimator. It is worth mentioning that the cost estimation in Galvatron-BMW considers the GPU performance slowdown from computation and communication overlapping, which has been ignored for a long time in previous approaches. We provide an implementation of Galvatron-BMW over PyTorch. Unlike existing toolbox-like systems (e.g., DeepSpeed [33], Megatron [21]) relying on users’ expertise and significant tuning efforts, Galvatron-BMW’s automatic parallelism only requires a few lines’ modifications on the original training script. Our evaluation selects four representative Transformers, including both NLP (i.e., BERT and T5) and CV (i.e., ViT, Swin Transformer). The experiments show that Galvatron-BMW could significantly outperform the four pure parallelisms and existing automatic parallelisms with limited dimensions (i.e., DP+TP and DP+PP) under various device memory budgets.

We summarize our contributions as follows:

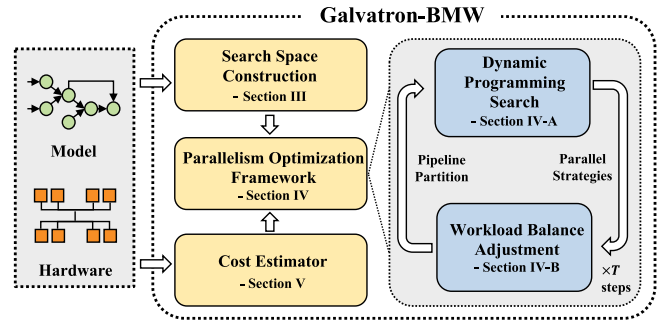


Fig. 1. System overview of Galvatron-BMW.

- 1) We enlarge the explored dimension of automatic parallelism for Transformer training to five parallelism dimensions, and introduce a novel decision-tree abstraction to decompose the large search space.
- 2) We design a novel parallelism optimization method to automatically find the most efficient hybrid parallelism strategy based on the estimated costs.
- 3) We consider both memory consumption and computation workload through a bi-objective optimization framework to maximize the hardware utilization during training.
- 4) We build Galvatron-BMW system that supports larger models' training and achieves up to 530% and 242% throughput speedups compared to state-of-the-art pure and hybrid parallelism methods respectively.

Fig. 1 shows the system overview of Galvatron-BMW, which takes the model and hardware environment as inputs, and comprises three primary modules: search space construction (Section III), parallelism optimization framework (Section IV), and cost estimator (Section V). In Section III, we introduce the search space construction of Galvatron-BMW with decision-tree-based decomposition. In Section IV, we propose our parallelism optimization framework, which leverages dynamic programming search and workload balance adjustment techniques to iteratively refine the optimal parallelism strategy. In Section V, we provide a cost estimator to estimate the execution cost and memory consumption efficiently and accurately. We also provide implementation details in Section VI and comprehensive experimental results in Section VII.

II. PRELIMINARY

A. Transformer Models

Transformers are first proposed to solve sequence modeling and transduction problems such as language modeling and machine translation [3]. The self-attention and point-wise feed-forward modules are the basic components in each Transformer layer. Most operations are dense algebras like matrix multiplications, resulting in huge computation costs and memory consumption.

Transformers in NLP: Different manners of using Transformer layers in NLP incur three mainly Transformer architectures, including encoder-only (for text classification, e.g.,

BERT and RoBERTa [34]), decoder-only (for text generation, e.g., GPT-2 and Transformer-XL [35]), and encoder-decoder (for sequence-to-sequence tasks, e.g., T5 and BART [36]). They have similar basic model components and some slight differences on the structures. For example, the decoder has an additional self-attention layer compared to the encoder. What's more, the encoder-decoder architecture combines encoders and decoders symmetrically (i.e., the same number of layers) together. These differences bring some distinct system workload characteristics in both computation and memory.

Transformers in CV: Transformers are also becoming increasingly attractive in computer vision areas. Vision Transformer (ViT) first replaces the tokens in languages with patches in images and the patches are fed into the encoder for the image classification task. Standard ViTs have a fixed number of patches and the same hidden dimension across different layers. Swin Transformer proposes a multi-stage hierarchical architecture with a shifted window-based attention to encode multi-scale patches. However, such multi-scale architectures also cause uneven computation and memory across layers.

B. Parallelism in Distributed Training

Data Parallelism: Data parallelism approaches are widely used to scale up the distributed training for large input datasets. It refers to distributing the data samples across multiple workers to compute and synchronize the model updates (e.g., gradients). Each worker should maintain a replica of the model which implies that the model should be fit into the device memory. To alleviate the redundant memory consumption, DeepSpeed ZeRO [32] (also named FSDP in FairScale [37]) has been proposed to partition the model states instead of replicating them. It is similar to model parallelism but still follows the data parallelism computation process except involving additional communications to share the model states.

Model Parallelism: Model parallelism divides the model into multiple parts and each worker is only responsible for the computation of the partial model. Due to the complexity of DL model architectures, a variety of model parallelism approaches have been proposed with different model partition techniques. There are mainly two kinds of paradigms commonly used for large-scale Transformers training, including distributed tensor parallelism (TP) and layer-wise pipeline parallelism (PP). For example, Megatron-LM [21] uses TP, which partitions the feed-forward and self-attention modules in Transformers to multiple devices and inserts communication operations (e.g., All-Reduce) to guarantee consistent results. GPipe [22] first proposes PP, treats each model as a sequence of layers and partitions the model into multiple composite layers across the devices. The workers are organized as a pipeline and transfer intermediate results at the partition boundaries between neighboring partitions. It further splits the mini-batch into smaller micro-batches to reduce the bubbles (i.e., idle time). PipeDream [24] and 1F1B-Flush [38] (also referred to as PipeDream-Flush) are also popular schedules for PP execution. In Galvatron-BMW, we default to 1F1B-Flush for its advantages of synchronous weight updates and demonstrating the same theoretical bubble rate as

GPipe, while being much more memory-efficient. However, 1F1B-Flush causes distinct memory cost across different PP stages, where shallower stages consumes more memory. Such memory workload imbalance can potentially impede system performance, necessitating careful optimization of PP workload balance.

Activation Checkpointing: Activation checkpointing (CKPT) is a commonly used technique in large-scale model training, which trades off computation for memory overhead. In the standard training procedure, all intermediate activations computed in forward propagation must be stored for gradient computation, which can be quite memory-intensive for large models. To alleviate the memory overheads, activation checkpointing divides the model into segments and only stores the input activations of each segment during forward pass, discarding the other intermediate results. As a trade-off, these discarded results need to be recomputed during backward propagation. Compared to other parallelisms that distribute the memory burden across physical devices, activation checkpointing distributes the memory burden across time dimension by delaying the memory consumption of most layers' intermediate activations to the backward pass with lower peak memory pressure. Therefore, we treat activation checkpointing as a special parallelism dimension that can be combined with other parallelisms naturally.

Automatic Parallelism: Recent approaches propose to integrate both data and model parallelism and search for better distributed training strategies. For example, FlexFlow, OptCNN, Tofu and TensorOpt consider both tensor parallelism and data parallelism. PipeDream and DAPPLE extend pipeline parallelism and enable data parallelism to replicate each pipeline stage. However, these approaches only explore the combination of data parallelism and at most one single model parallelism dimension. Such limited decision spaces cannot generate efficient enough parallelization plan for many workloads. In fact, industrial companies have taken great efforts to explore better parallelism combinations when training large Transformers on their clusters, such as Turing-NLG [39] from Microsoft and GPT-3 [7] from OpenAI. These evidences suggest that it is necessary to design an automatic parallelization system covering as many parallelism decisions as possible, without relying on strong system tuning experience from human experts.

III. SEARCH SPACE CONSTRUCTION

The goal of Galvatron-BMW is to automatically search within the composite parallelism space and generate the optimal parallelization plan for the given Transformer model and the distributed environment. The key challenge comes from the large search space when considering multiple parallelism strategies and making fine-grained decisions for the model parameters. In this section, we first introduce the search space of Galvatron-BMW and propose our decision-tree-based decomposition to explore the search space more efficiently.

A. Search Space Analysis

We first analyze the overhead of training Transformer models, and take an example environment with two GPUs to better

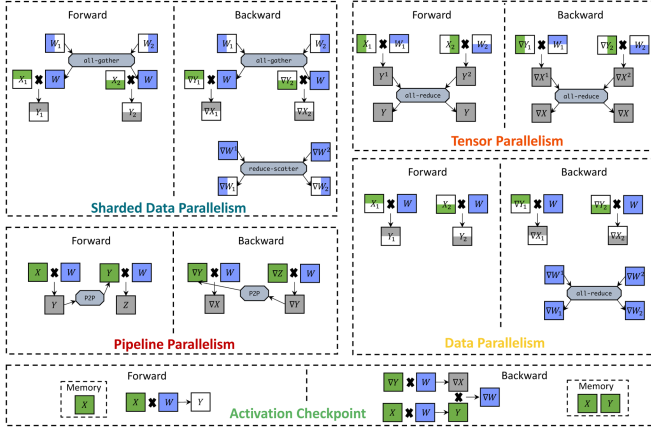


Fig. 2. Illustration of different basic parallelisms in Galvatron-BMW. We use the green and gray colors to denote the input and output activations for both forward and backward computation. The model parameters and gradients are in blue.

illustrate the large search space, optimization target, and necessary constraints. Then we extend the problem to multi-GPU cases.

1) *Overhead Analysis*: A Transformer model can be treated as a sequence of L layers, and each layer L_i contains a set of model parameters $\mathbf{w}_i$, along with their corresponding parameter gradients and optimizer states, and the combination of these three is referred to as the model states $\mathbf{ms}_i$. Due to the back propagation, the forward computation results (i.e., activations) $\mathbf{f}_i$ should be kept inside the device memory. Specifically, activations $\mathbf{f}_i$ include boundary activations $\mathbf{bnd}_i$, which are the input tensors of each layer, and the intermediate activations $\mathbf{int}_i$, which is the intermediate results inside of each layer, and it depends on the parallelism strategy that which part of $\mathbf{f}_i$ requires stashing. Besides $\mathbf{f}_i$, the calculation of gradients during back propagation may require extra backward activations $\mathbf{b}_i$.

The problem is to select the optimal parallelism strategy for each layer individually from a large search space, which is a composition of DP, SDP, PP, TP and CKPT. As illustrated in Fig. 2, all these parallelism strategies could split the computation workloads into multiple physical devices or distribute memory burden across time dimension, but they have distinct memory consumption and communication overheads, finally leading to different system efficiency. The overall computation overhead and communication overhead of the Transformer model are the summation of the layer overhead, while the overall memory consumption E_{all} is calculated by formula (1), where $c(\cdot)$ is the memory cost function.

$$E_{all} = \max_{i=1}^L \{ \sum_{k=1}^i c(\mathbf{f}_k) + c(\mathbf{b}_i) + \sum_{k=1}^L c(\mathbf{ms}_k) \}. \quad (1)$$

Specifically, when conducting back propagation of layer i , the peak memory cost is the summation cost of activation $\mathbf{f}_1, \dots, \mathbf{f}_i$ and $\mathbf{b}_i$, as well as model states $\mathbf{ms}_1, \dots, \mathbf{ms}_L$. And E_{all} takes the maximum of the peak memory cost of each layer.

2) *Two-GPU Example*: Considering for a single layer L_i in the model, we analyze its costs under different parallelism strategies on two GPUs as follows.

Data Parallelism: In DP, each GPU has a model replica and half of the input data samples. Since the size of activations is proportional to the number of data samples, each GPU only needs to store half of the forward activations. After the backward computation, the GPUs should synchronize their gradients (i.e., all-reduce) before updating the model, which has the same size as model parameters.

Sharded Data Parallelism: In SDP, each GPU has half of model parameters and half of the input data samples. However, it requires two times all-gather to collect the sharded model parameters for forward and backward computation and once reduce-scatter to update gradients. Since an all-reduce operation is equivalent to the combination of once all-gather and once reduce-scatter, the communication cost of SDP is $1.5 \times$ larger than DP.

Pipeline Parallelism: In PP, the layer L_i could be placed on either GPU 0 or GPU 1, resulting in two possible memory costs: (1, 0) and (0, 1). The communication cost is mainly determined by whether the neighboring layers are on the same device. In practice, each device may have a sub-sequence of layers and only the activations from the boundary layers should be transferred. The efficiency of PP is also affected by the pipeline bubbles (i.e., idle time), which can be reduced by splitting micro-batches. Besides, the workload balance also affects its system efficiency, where the model stage with the highest execution cost or the highest memory consumption usually becomes the bottleneck.

Tensor Parallelism: In TP, each GPU also has half of model parameters. Unlike SDP, TP allows each device to perform the forward computation (e.g., matrix multiplications and self-attentions) with half model. It requires to synchronize the activations with the all-reduce operations for both forward and backward computation. Due to the intermediate synchronization, TP has some additional replications of the activations.

Activation Checkpointing: Unlike other parallelisms that split computation and memory workload over physical devices, CKPT distributes the memory overhead across the time dimension, and brings no communication cost but extra computation cost. It enables the release of intermediate results $\mathbf{int}_i$ during the forward pass and only stores boundary activations $\mathbf{bnd}_i$, which cuts down the forward memory consumption, but requires additional forward computation to recompute $\mathbf{int}_i$ during backward propagation, and brings backward memory consumption as well. Specifically, for layers applying CKPT, $c(\mathbf{f}_i) = c(\mathbf{bnd}_i)$ and $c(\mathbf{b}_i) = c(\mathbf{int}_i)$, while for others not applying CKPT, $c(\mathbf{f}_i) = c(\mathbf{bnd}_i) + c(\mathbf{int}_i)$ and $c(\mathbf{b}_i) = 0$.

3) *Multi-GPU Extension*: When extending to multi-GPU, the problem becomes more complicated since even a single layer could have a variety of hybrid parallelism strategies by integrating multiple parallelism paradigms. For example, for two nodes with 4 GPUs in total, it's easy to integrate 2-way TP within a node and 2-way PP across nodes. Alternatively, using 2-way PP within a node and 2-way DP across nodes is also possible. Besides, CKPT can also be combined with other parallelisms, and may bring additional communication overhead

, as it requires an extra forward pass before calculating gradients. For example, when combined with TP, CKPT requires extra all-reduce operations during recomputation. In the 4-GPU example, it's possible to use 2-way TP and 2-way PP without CKPT, and also valid to use 2-way TP and 2-way PP with CKPT, where CKPT trades-off between memory consumption and time cost. Therefore, CKPT doubles the size of search space further. Moreover, when scaling to 8 GPUs or even more GPUs, there exist hundreds of candidate strategies for a single layer. For a given model, the entire search space is much larger and exponentially growing with the number of layers.

B. Decision-Tree-Based Search Space Decomposition

Considering for such a large search space, it is impossible to brute-force search all the combinations of these parallelism paradigms within a feasible time budget. Therefore, to explore the search space more efficiently, we introduce the following key intuitions from empirical observations or theoretical analysis.

Takeaway #1: PP prefers to be applied across device “islands”. Each island is a set of devices with higher-bandwidth interconnects (e.g., NVLink, PCIe) and should be in charge of a stage in the pipeline. Compared to other parallelisms, PP has much less communication overheads especially for large models. Because each stage typically has multiple layers but only requires to communicate the activations from the boundary layers. It is sensible to perform PP partition first across slower inter-island links (e.g., QPI, Ethernet).

Takeaway #2: Suppose the devices are homogeneous, these parallelism strategies prefer to divide the devices into groups with equal size. For example, a 2-way DP on 4 GPUs means two 2-GPU groups, rather than a single GPU and one 3-GPU group. Consequently, the optimal hybrid parallelism strategy on one group should be also consistent with those of the other groups. Note that, it could fail for PP since the model partitions may have different computation operations, resulting in different optimal parallelism strategies.

Based on the above important intuitions, we design a decision-tree to decompose the search space and represent the candidate hybrid parallelism strategies. We next present the details of constructing the decision-tree.

Insights Underpinning Decision-Tree: We find that most existing automatic parallelism approaches only involve two parallelism dimensions (e.g., OptCNN and FlexFlow), which is easily to enumerate all possible parallelism configurations for a single layer. After involving pipeline parallelism (e.g., PipeDream), they often partition the model into different stages first and each stage is then assigned to a subset of devices. Such kind of observation suggests us to explore the hierarchical search space by utilizing a decision-tree. Another motivation is that we need the tree structure to capture the orders when applying parallelism even inside a stage. Due to the device topology and hierarchical bandwidth, it is necessary to consider the permutations of hybrid strategies since they may have different communication efficiencies.

Decision-Tree Construction: Given a Transformer model, Galvatron-BMW first applies PP to partition the model into

multiple stages. In the meanwhile, the devices are also divided into multiple groups with the same size. As suggested by Takeaway #1, it prefers grouping between devices with higher bandwidth. For an 8-GPU scenario, Galvatron-BMW will attempt 1/2/4/8-way PP respectively. Suppose the model is partitioned evenly by PP, based on Takeaway #2, the size of the corresponding device group should be 8/4/2/1 respectively after applying PP, which directly determines the number of leaf nodes in our decision-trees. As shown in Fig. 3, given the number of leaf nodes, there might exist multiple possible tree structures. We define the decision-tree construction rules as follows:

- Each decision-tree denotes a sub-search-space and its height represents the number of available parallelism paradigms including DP, TP, PP and SDP.
- Any one of DP, TP, PP and SDP cannot be applied repeatedly in different levels of a decision-tree.
- The degree of non-leaf nodes should be selected from $\{2, 4, 8, \dots\}$.
- Each decision-tree can be decided to apply CKPT (S'_i) or not to apply CKPT (S_i).

With the above rules, the constructed trees could represent the arbitrary combinations of these parallelisms in a non-overlap manner. The guidance from *Takeaway #1* and *#2* significantly helps Galvatron-BMW to avoid the unnecessary and inefficient parallelism combinations. For a single layer with 8 GPUs, it produces 68 different hybrid parallelism strategies, which reduces the original combinational search space including hundreds of strategies by one order of magnitude. It could be further optimized as follows:

Takeaway #3: Using SDP is always better than integrating DP and SDP. We make a comparison with N -way DP, N -way SDP, and the combination of N_1 -way DP and N_2 -way SDP ($N_1 \times N_2 = N$). First, SDP always has fewer model parameters than DP+SDP since $N_2 \leq N$. Second, integrating DP and SDP will lead to two rounds of communication including $2(N_1 - 1)/N_1$ for N_1 -way DP and $3(N_2 - 1)/N_2$ for N_2 -way SDP. Given $N_1 \times N_2 = N$, we can prove that the minimum value of its cost is still larger than that of pure SDP. Therefore, we exclude such combinations from our search space. After applying *Takeaway #3*, we could further reduce the number of candidate strategies to 44 for a single layer with 8-GPUs.

IV. PARALLELISM OPTIMIZATION FRAMEWORK

The target of Galvatron-BMW is to generate the optimal hybrid parallelism strategy for the input DL model with the given devices.

Problem Formulation: We define the optimization problem in Galvatron-BMW as follows. Given model M (with L layers) and N devices (with memory capacity of E), the object is to find the largest throughput T_{pt} and return the corresponding parallelism strategy, which is made up of the fine-grained layer-level parallelism strategies. According to the decision-tree-based decomposition proposed in Section III-B, PP is first applied to partition the model into multiple stages, and to divide the devices into multiple device groups. Subsequently, the partitioned model stages are assigned to the corresponding device groups, and

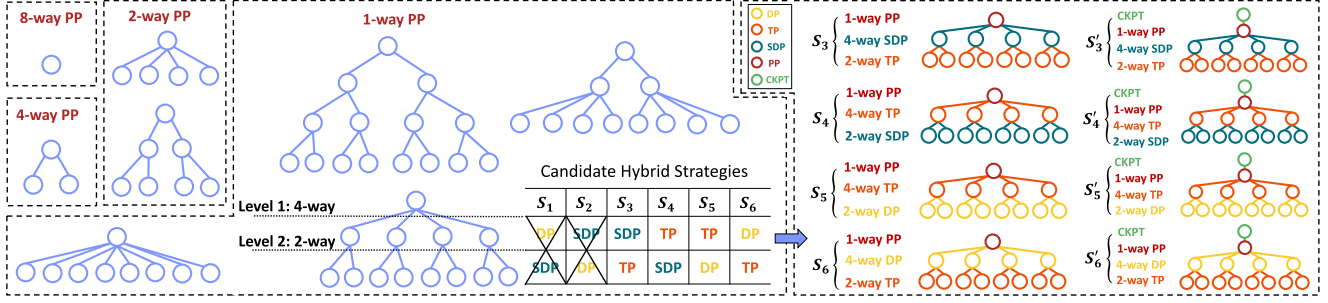


Fig. 3. Illustration of the decision trees for 8 GPUs under different PP degrees (i.e., 8/4/2/1). We select one of them to introduce how to use the tree to describe the candidate hybrid parallelism strategies. We remove S_1 and S_2 as suggested by *Takeaway #3* and illustrate the other hybrid strategies on the right part. Each decision tree can be decided to apply CKPT (S'_3 - S'_6) or not to apply CKPT (S_3 - S_6). In total, there are 44 candidate hybrid strategies for all trees.

optimization of other parallelism dimensions is conducted for each model stage. It is evident that pipeline partition plans influence the workload on each device group, thereby affecting the optimization outcome. Therefore, we address the optimization problem in two folds:

Question 1: Given an ideally balanced pipeline partition plan, how to conduct parallelism optimization for each model stage and find the optimal hybrid parallelism strategies? (Section IV-A, Galvatron-Base)

Question 2: How to find an optimal pipeline partition plan, which balances the pipeline workload (including both memory and computation) and maximizes the system throughput? (Section IV-B, Galvatron-BMW)

A. Basic Parallelism Optimization

1) *Basic Optimization Workflow:* We first propose Galvatron-Base to solve *Question 1* and the optimization algorithm workflow is illustrated in Algorithm 1. Basically, the system throughput equals to the ratio between the batch size and the iteration time (i.e., per-batch execution time). Tuning the batch size could lead to distinct memory consumption, computation costs and communication overheads. Scaling the model training with hybrid parallelism strategies could reduce the memory consumption and enlarge the batch size. But it could also bring significant communication overheads. In other words, the highest training throughput does not have to come with the largest batch size. Therefore, in Galvatron-Base, we gradually increase the explored batch size (line 2) and keep tracking the maximum system throughput until exceeding the device memory for all possible parallelism strategies (lines 11-15).

Given a candidate batch size B , Galvatron-Base then utilizes *Takeaway #1* to apply PP at first. We suppose the total number of devices N is the power of two (e.g., 4, 8, 16), which is common in dedicated GPU training clusters. So we only explore the 2th powered PP degrees (line 4). In default, for each unique PP degree P , we assume its pipeline partition plan is given (line 5) and represented as an array $\mathbf{p}$, where each item $\mathbf{p}_i$ denotes the number of model layers for the i th pipeline stage. For example, $\mathbf{p} = [12, 12]$ indicates that a 24-layer model is partitioned into two stages with 12 layers each. Note that, the devices are evenly divided into P groups and the model is also partitioned into P

stages (line 6), guided by several load balancing factors (e.g., the number of layers/parameters, the maximum memory usage, and the execution time). We leave these further discussion to Section IV-B. Then we conduct the Galvatron_Search function to optimize parallel strategies (line 7). In Galvatron_Search (line 17), we first initialize the micro-batch number m (line 18) and calculate the micro-batch size B_m . For each model stage, we construct the decision tree that represents the candidate hybrid parallelism strategies composed of DP, SDP, TP, and CKPT. After obtaining the strategies set S , we further determine the parallelization plan for each layer in M_i under the limited device memory budget E with the dynamic programming search algorithm (Section IV-A2). The search results are used to calculate the whole pipeline execution time (line 27) based on the cost model in Section V.

2) *Dynamic Programming Search:* For a given model stage including L layers, we suppose the function $C(L, E)$ represents the total execution time of these L layers under the device memory budget E . When applying any parallelism strategy $S_j \in S$, we define $c(L, S_j)$ to denote the execution time, $O_f(L, S_j)$ and $O_b(L, S_j)$ to represent the memory consumption of forward activations $\mathbf{f}_L$ and backward activations $\mathbf{b}_L$, and $O_{ms}(L, S_j)$ to denote the memory consumption of model states $\mathbf{ms}_L$. The overall memory consumption of L layers $E_{all}(L)$ is calculated by (2). We also calculate the total forward memory consumption of L layers $E_f(L)$ by (3), which includes the consumption of forward activations and model states. It is obvious that $E_f(L) \leq E_{all}(L)$.

$$E_{all}(L) = \max_{i=1}^L \{ \sum_{k=1}^i O_f(k, S_{j_k}) + O_b(i, S_{j_k}) + \sum_{k=1}^L O_{ms}(k, S_{j_k}) \} \quad (2)$$

$$E_f(L) = \sum_{i=1}^L \{ O_f(i, S_{j_i}) + O_{ms}(i, S_{j_i}) \}. \quad (3)$$

We aim to optimize $C(L, E)$ given the memory constraint $E_{all}(L) \leq E$ using dynamic programming search. However, we find that due to the maximum operation in (2), two memory states need to be stored during state transition, which leads to a quadratic complexity in terms of memory constraint E (refer to Appendix A1 for details), available online. This is unacceptable in practice. To ensure the linear complexity with respect to memory, we decouple the forward memory $E_f(L)$ from $E_{all}(L)$, and

Algorithm 1: Galvatron-Base Optimization.

Input: model: M , #devices: N , device memory: E
Output: maximum system throughput T_{pt}

```

1  $T_{pt} \leftarrow 0$ ;
2 for Batch size  $B \leftarrow 1, 2, \dots$  do
3   Time costs set  $\mathcal{C} \leftarrow \{\}$ ;
4   for PP degree  $P \in \{1, 2, 4, 8, \dots, N\}$  do
5      $\mathbf{p} \leftarrow \text{PP\_Partition\_Init}(M, P)$ ;
6     Model stages  $\{M_i\}_{i=1}^P \leftarrow \text{Model\_Partition}(M, \mathbf{p})$ ;
7      $C, \mathcal{S} \leftarrow \text{Galvatron\_Search}(E, \{M_i\}, N, B, P)$ ;
8      $\mathcal{C}.append(C)$ ;
9   end
10   $C_{opt} \leftarrow \min(\mathcal{C})$ ;
11  if  $C_{opt}$  is not  $\infty$  then
12     $T_{pt} \leftarrow \max(B/C_{opt}, T_{pt})$ ;
13  else
14    return  $T_{pt}$ ;
15  end
16 end
17 Function  $\text{Galvatron\_Search}(E, \{M_i\}, N, B, P)$ :
18    $m \leftarrow \text{Init\_Microbatch\_Num}(\{M_i\}, B, P)$ ;
19    $B_m \leftarrow B/m$ ;
20   Strategies set  $\mathcal{S} \leftarrow \text{Construct\_Decision\_Tree}(N/P)$ ;
21   Model strategies list  $\mathcal{S} \leftarrow []$ ;
22   Model cost  $C \leftarrow 0$ ;
23   for  $i \in \{1, 2, \dots, P\}$  do
24      $C_i, \mathcal{S}_i \leftarrow \text{Dynamic\_Programming}(E, M_i, B_m, \mathcal{S})$ ;
25      $\mathcal{S}.extend(\mathcal{S}_i)$ ;
26   end
27    $C \leftarrow \text{Pipeline\_Cost}(\{C_i\}_{i=1}^P, m)$ ;
28   return  $C, \mathcal{S}$ ;

```

first optimize $C(L, E_{fwd})$ with the forward memory constraint $E_f(L) \leq E_{fwd}$, where $E_{fwd} \leq E$, and finally check the validity of the overall memory, i.e., $E_{all}(L) \leq E$.

First, we discuss how to optimize $C(L, E_{fwd})$ with constraint $E_f(L) \leq E_{fwd}$ using dynamic programming. Since the problem follows the optimal substructure property (refer to Appendix A2 for detailed proof), available online, we start with $C(0, \cdot) = 0$ and $C(\cdot, 0) = \infty$, then we can derive the following state transition formula:

$$C(L, E_{fwd}) = \min_{S_j \in \mathcal{S}} \{C(L-1, E_{fwd} - O_f(L, S_j) - O_{ms}(L, S_j)) + c(L, S_j) + R(L, S_i, S_j)\}, \quad (4)$$

where $R(L, S_i, S_j)$ is the transformation cost between the L th layer applying S_j and its former layer applying S_i . If two neighboring layers have different parallelism strategies, the former layer's output should be transformed to the required data layout to facilitate the next layer's parallelism. For example, if the former layer uses the combination between 2-way DP and 2-way TP and the current layer attempts to use 4-way DP, a transformation step is necessary to prepare the full model replica and the 1/4 forward activation at each device for the current layer. During the state transition process, if the forward memory usage exceeds the budget E_{fwd} , the cost function C should return infinity.

Second, based on the state transition formula in (4), we introduce our overall dynamic programming algorithm (the detailed algorithm is given in Appendix A3), available online, which ensures $E_{all}(L) \leq E$. Given a device memory budget E , we use E_{fwd} ($E_{fwd} \leq E$) as the forward memory budget, and the rest part $E - E_{fwd}$ is spared for the backward peak memory $E_b(L) = E_{all}(L) - E_f(L)$. To maximize the memory utility, we gradually increase and traverse E_{fwd} to optimize $C(L, E_{fwd})$ using (4), and then check the validity of the overall memory with the searched strategies, i.e., $E_{all}(L) \leq E$. Finally, we find the largest forward memory budget E_{fwd}^{opt} with its searched strategies satisfying $E_{all}(L) \leq E$, and the optimized throughput $C(L, E_{fwd}^{opt})$ is the final output.

3) *Complexity Analysis:* The proposed dynamic programming search formula in (4) has a computation complexity of $\mathcal{O}(LE|S|)$. As we can see, the size of the single layer's decision space is crucial for the entire complexity and our proposed decision-tree significantly reduces the space and makes it feasible. The number of layers L and the memory budget E also affect the complexity. For extreme cases with thousands of layers or huge memory capacity, we can further reduce the complexity by taking coarse-grained explorations, e.g., fusing multiple layers, using large memory granularity.

B. Bi-Objective Optimization of Workload Balance

We assume an ideally balanced pipeline partition in Galvatron-Base which may not be true in practice. In fact, perfect workload balance could be difficult to achieve due to several reasons. For example, the 1F1B-Flush pipeline scheduling brings distinct memory consumption for different model stages (Section II-B). Besides, some model structures (e.g., encoder-decoder) are naturally heterogeneous and hardly to be evenly partitioned based on any single empirical guideline. For instance, the decoder usually has much shorter sequence length than the encoder, leading to workload imbalance of both memory and execution time. Therefore, to address *Question 2*, we further propose Galvatron-BMW that considers the **B**alancing trade-off between **M**emory **W**orkload and computation workload.

1) *Pipeline Workload Balance:* When training a model M in PP (i.e., no matter using GPipe or 1F1B-flush) with mini-batch size B and memory budget E for each worker, the overall time cost $C(M, B)$ and peak memory cost $O(M, B)$ can be represented as follows:

$$C(M, B) = (m-1) * \max_{i=1}^P C(M_i, B_m) + \sum_{i=1}^P C(M_i, B_m)$$

$$O(M, B) = \max_{i=1}^P O(M_i, B_m) \leq E, \quad (5)$$

where $C(M_i, B_m)$ and $O(M_i, B_m)$ denotes the execution time and the memory consumption of the i^{th} model stage M_i under micro-batch size B_m ($B_m = B/m$). As can be seen, the overall pipeline execution time largely depends on the slowest worker, and the memory bottleneck lies in the worker with the heaviest memory workload. To quantify the workload imbalance, we

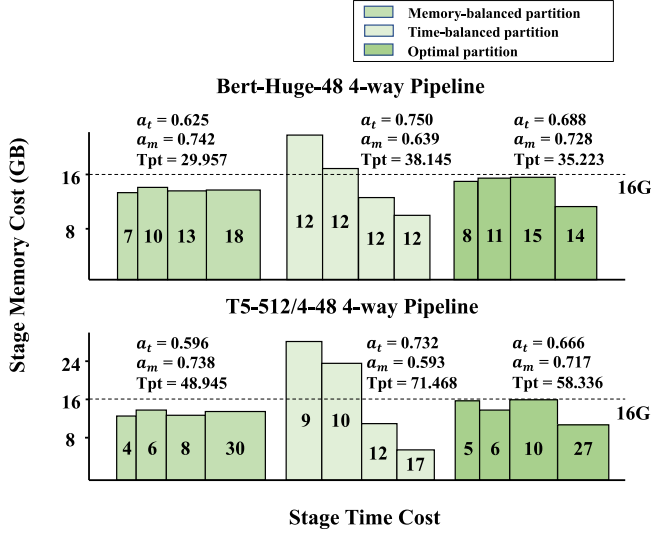


Fig. 4. Performance of 4-way 1F1B-Flush pipelines with different partition plans on A100 GPUs. The global batch size is 32 for BERT-Huge-48 and 64 for T5-512/4-48 (see more details of the model in Section VII-A), and the micro-batch number is 8. Bars (from left to right) symbolize pipeline stage 1 through 4: height for memory consumption, width for time cost (normalized), including the number of layers, balance degrees and throughput.

define the following time and memory balance degrees:

$$\alpha_t = 1 - \frac{\max_{i=1}^P C(M_i, B_m)}{\sum_{i=1}^P C(M_i, B_m)}, \alpha_m = 1 - \frac{\max_{i=1}^P O(M_i, B_m)}{\sum_{i=1}^P O(M_i, B_m)}, \quad (6)$$

which satisfy that $0 \leq \alpha_t, \alpha_m \leq 1 - \frac{1}{P}$. It's easy to observe that a larger balance degree implies lower total execution time or peak memory usage. Specially, $\alpha_t = 1 - \frac{1}{P}$ or $\alpha_m = 1 - \frac{1}{P}$ indicate a perfect workload balance of time or memory.

Fig. 4 shows two examples of 4-way 1F1B-Flush PP, implementing both memory- and time-balanced partition. The time-balanced pipeline exhibits similar execution costs across all stages, resulting in a quite balanced α_t , which is close to $1 - \frac{1}{4} = 0.75$ for both models. Nonetheless, its memory consumption is seriously imbalanced due to the influence of 1F1B scheduling, leading to a relatively low α_m , where deeper stages consuming considerably less memory than shallower ones. In contrast, the memory-balanced pipeline demonstrates nearly uniform memory consumption across all stages, resulting in a substantial α_m . But it allocates more layers to deeper stages for memory balance, increase their execution costs, and reduces α_t . Regarding system efficiency, a greater α_t corresponds to increased throughput for time-balanced pipelines. However, under a limited memory budget (e.g., 16 GB), some stages of time-balanced pipelines are prone to memory exhaustion, making it unfeasible. Inversely, while the memory-balanced pipelines yield lower system throughput, their higher α_m ensures balanced memory budget utilization, thereby facilitating the training process. Hence, our objective is to devise a pipeline partition plan that optimizes both α_m and α_t , to ensure workload balance and enhance system efficiency.

Based on these two balance degrees, we can further define $\mathbf{p}_t$ and $\mathbf{p}_m$ as the extremely balanced time and memory-balanced

Algorithm 2: Galvatron-BMW Bi-Objective Optimization.

Input: model: M , #devices: N , device memory: E , batch size: B_0 , constant: r

Output: maximum system throughput T_{pt}

```

1  $T_{pt} \leftarrow 0$ ;
2 Queue  $Q \leftarrow \emptyset$ ;
3 for Batch size  $B \in [B_0 - r, B_0 + r]$  do
4   Time costs set  $\mathcal{C} \leftarrow \{\}$ ;
5   for PP degree  $P \in \{2, 4, 8, \dots, N\}$  do
6      $S_0 \leftarrow \text{Strategy\_Init}()$ ;
7      $\mathbf{p}_0 \leftarrow \text{PP\_Partition\_Init}(M, P, S_0)$ ;
8      $Q.\text{push}(\mathbf{p}_0)$ ;
9     while  $Q \neq \emptyset$  do
10      Pipeline partition  $\mathbf{p} \leftarrow Q.\text{pop}()$ ;
11      Model stages
12       $\{M_i\}_{i=1}^P \leftarrow \text{Model\_Partition}(M, \mathbf{p})$ ;
13       $C, S \leftarrow \text{Galvatron\_Search}(E, \{M_i\}, N, B, P)$ ;
14       $\mathbf{p}' \leftarrow \text{PP\_Partition\_Adjust}(\mathbf{p}, M, P, S)$ ;
15      if  $\text{Validate}(\mathbf{p}', M, P, S)$  then
16         $Q.\text{push}(\mathbf{p}')$ ;
17      end
18    end
19     $\mathcal{C}.\text{append}(C)$ ;
20  end
21   $C_{opt} \leftarrow \min(\mathcal{C})$ ;
22  if  $C_{opt}$  is not  $\infty$  then
23     $T_{pt} \leftarrow \max(B/C_{opt}, T_{pt})$ ;
24  else
25    return  $T_{pt}$ ;
26 end
```

partition plans, satisfying $\alpha_t(\mathbf{p}_t) \geq \alpha_t(\mathbf{p}), \alpha_m(\mathbf{p}_m) \geq \alpha_m(\mathbf{p})$ for any partition plan $\mathbf{p}$. We can infer that the optimal partition $\mathbf{p}^*$ should guarantee

$$\alpha_t(\mathbf{p}_m) \leq \alpha_t(\mathbf{p}^*) \leq \alpha_t(\mathbf{p}_t),$$

$$\alpha_m(\mathbf{p}_t) \leq \alpha_m(\mathbf{p}^*) \leq \alpha_m(\mathbf{p}_m). \quad (7)$$

Because for any partition $\mathbf{p}$, if $\alpha_t(\mathbf{p}) < \alpha_t(\mathbf{p}_m)$, we'd rather choose memory-balanced partition $\mathbf{p}_m$ as it has both better memory and speed balance, and if $\alpha_m(\mathbf{p}) < \alpha_m(\mathbf{p}_t)$, we prefer $\mathbf{p}_t$ for the same reason. Taking the optimal partition in Fig. 4 as an example, it effectively mitigates the memory exhaustion observed in time-balanced pipelines (i.e., with median values of α_t and α_m) and simultaneously boosts the throughput of memory-balanced pipelines by as much as 19.2%. However, it is unlikely to find $\mathbf{p}^*$ by optimizing these two co-related balance degrees separately.

2) *Bi-Objective Optimization Workflow:* To achieve balanced workloads across pipeline stages and maximize the system performance, we propose the bi-objective optimization workflow of Galvatron-BMW in Algorithm 2. It basically follows the workflow of Algorithm 1 and reuses the Galvatron_Search function. The key difference is that we adjust the pipeline partition iteratively to optimize the workload balance simultaneously. For each iteration, we start from a pipeline partition $\mathbf{p}$ (line 10) that is initialized (line 7) by the memory-balanced partition $\mathbf{p}_m$. We then conduct Galvatron_Search to optimize the

parallel strategies $\mathcal{S}$ based on $\mathbf{p}$. Since the update of $\mathcal{S}$ might change the pipeline workload balance, we need to adjust the pipeline partition $\mathbf{p}$ correspondingly.

Here we provide a heuristic partition adjustment method that greedily cuts down the workload of the slowest pipeline stage, and adjust the pipeline from memory-balanced to time-balanced. In particular, given an input partition plan $\mathbf{p}$, we first find the model stage with the maximum time cost $C_{\max}$. Then we move its boundary layer to its adjacent stages and obtains a new partition $\mathbf{p}'$. The validation function (line 14) prevents the opportunistic adjustment from being harmful to the overall system efficiency by adding limitations on $\mathbf{p}'$, including 1) the time costs of model stages should be no more than the previous maximum cost $C_{\max}$, 2) the memory costs of model stages should not exceed memory budget, and 3) the memory costs of model stages should not surpass the maximum stage memory cost under partition $\mathbf{p}_t$. If all criteria are met, it can be demonstrated that $\mathbf{p}'$ satisfies condition 7 and $\alpha_t(\mathbf{p}') \geq \alpha_t(\mathbf{p})$, indicating $\mathbf{p}'$ is superior to the prior partition $\mathbf{p}$ in time balance (refer to Appendix B for more details), available online. Then partition $\mathbf{p}'$ is considered a feasible intermediate partition and is pushed to queue Q for subsequent search iterations.

V. COST ESTIMATOR

Galvatron-BMW provides a cost estimator to estimate the computation and communication costs and memory consumption during the optimization process. Current methods primarily use *profiling* or *simulating* for estimation. In Galvatron-BMW, we take advantages from both sides and design a cost model to make the estimations cheap, efficient and accurate. Specifically, for the memory consumption, we use the shape of a tensor and its data type to calculate its memory. For the computation time, we suppose it could be estimated by the product of the batch size and the per-sample computation time. The latter could be measured by profiling the real layer execution time on a single device. Note that, the Transformers are mainly composed by matrix multiplication operations, so the backward computation is usually twice of the forward computation. For the communication time, we can approximate it by using the size of tensor to be transferred divided by the inter-device connection's bandwidth.

With the above computation and communication cost estimations, $c(l, s)$ (i.e., the cost of a given layer l using a specific parallelism strategy $s \in \mathcal{S}$) could be calculated by simulating the execution process. It consists of two steps, e.g., forward and backward computation. The simulation for the forward computation is simple and directly sums up the computation and communication costs (i.e., `all-gather` in SDP and `all-reduce` in TP). However, during the backward process, DP and SDP enable the computation and communication overlapping, which may bring estimation errors. A typical choice is to take the maximum value from the computation and communication costs (e.g., PipeDream [24]). Existing automatic parallelism approaches barely notice that modern GPUs simultaneously performing compute kernels and communication primitives (e.g., NCCL [40]) lead to slowdown for both sides. The performance

degradation is mainly from the resource contention of thread warps in GPU streaming multiprocessors. We find that such contention could slow down the computation and communication by $1.3\times$ in our evaluations, which is consistent with some recent observations [41]. By considering the overlapping slowdown, Galvatron-BMW makes more accurate estimations and better optimizations. Besides, for layers applying CKPT, we add an extra forward propagation cost on $c(l, s)$ to simulate the recomputation overhead. By summing up the layer cost $c(l, s)$, we calculate the cost of pipeline model stage M_i , $C(M_i, B_m)$, where B_m is the micro-batch size. Then the overall time cost of the model is estimated according to (5), which considers the workload balance.

VI. IMPLEMENTATION

Galvatron-BMW is an automatic parallel training framework especially for Transformer models (open sourced at [42]), as a part of a novel distributed DL system Hetu [43], [44], [45], [46], [47]. We provide a simple and efficient interface to Galvatron-BMW users by making a few lines' modifications on the PyTorch training programs [48], [49].

Communication Group: We implement all communication primitives with PyTorch NCCL functions. As Galvatron-BMW supports complex hybrid parallelism strategies, there could exist many communication groups among the GPUs in the generated parallelization plan. To avoid the expensive NCCL groups construction overheads, Galvatron-BMW maintains a global communication group pool which is created in advance and contains all groups that might be used.

Transformation Optimization: We propose an efficient *Slice-Gather* step to perform the transformations automatically between two neighboring layers with different parallelism strategies. Given the previous layer with strategy A and the current layer with strategy B, the main idea of *Slice-Gather* is to ensure the input activations for the current layer are placed on the devices according to the requirement of strategy B, which has been extensively studied [50], [51].

VII. EXPERIMENTS

A. Experimental Setups

In this section, we compare Galvatron-BMW with 4 pure distributed parallelism strategies implemented by the state-of-the-art systems including PyTorch DDP [49] for DP, Megatron [21] for TP, PyTorch GPipe [52] for PP, and FairScale FSDP [53] (similar to DeepSpeed ZeRO Stage-3 [32]) for SDP. To help better understand the benefits of each technique in Galvatron-BMW, we also provide several auxiliary baselines for further comparisons. Apart from Galvatron-Base, we use Galvatron to represent a variant of Galvatron-Base that disables CKPT. Based on that, we further create Galvatron (DP+TP) and Galvatron (DP+PP) to verify the training efficiency of previous automatic parallelism approaches with limited parallelism dimensions (i.e., DP+TP and DP+PP). Galvatron (1F1B+Bi-obj) is based on Galvatron but enables bi-objective optimization (in other words, it can

TABLE I
STATISTICS OF MODELS

Model	Layer Num	Hidden Size	Param. Num	Acti. Size/sample
BERT-Huge-32	32	1280	672M	3149.39MB
BERT-Huge-48	48	1280	987M	4657.51MB
BERT-xHuge	128	2560	10.2B	24210.05MB
ViT-Huge-32	32	1280	632M	646.5MB
ViT-Huge-48	48	1280	947M	968.59MB
ViT-xHuge	128	2560	10.1B	5313.9MB
T5-Large-32	16 Enc.+16 Dec.	1024	502M	4119.66MB
T5-Large-48	24 Enc.+24 Dec.	1024	737M	6107.75MB
T5-512/4-32	16 Enc.+16 Dec.	1024	502M	1777.06MB
T5-512/4-48	24 Enc.+24 Dec.	1024	737M	2473.10MB
Swin-Huge-32	2/2/26/2	320/640/1280/2560	701M	726.59MB
Swin-Huge-48	2/2/42/2	320/640/1280/2560	1016M	1016.8MB
GPT3-15B	48	5120	15.4B	32889.04MB
GPT3-39B	48	8192	39.1B	58645.34MB
GPT3-65B	80	8192	64.9B	97557.98MB

be treated as Galvatron-BMW but disables CKPT). Specially, DeepSpeed 3D is an expert-designed baseline [54] integrating DP, TP, and PP globally. We select NLP models BERT, T5 as well as CV models ViT, Swin Transformer as our experimental models. The statistics of models are listed in Table I. We select the Adam optimizer and use the English Wikipedia and ImageNet-1 K as input datasets for them respectively. To further verify the performance of Galvatron-BMW, we also select an imbalanced model for some experiments, T5-512/4, a variant of T5 model for question answering task SQuAD, where the sequence length is 512 for encoders and 4 for decoders. Due to the shorter sequence length, the decoders have much less memory consumption than the encoder, leading to the memory imbalance. Most experiments are evaluated on a single node equipped with 8 Nvidia RTX TITAN 24 GB GPUs using PCIe 3.0. For PP, we manually tune the number of micro-batches to minimize the bubbles and estimate its costs. All results are averaged over 100 iterations.

B. End-to-End Comparison

Table II shows the overall system throughput results of different models under different strategies with different memory constraints, along with the corresponding batch size. As we can see, under different model scales and memory budgets, Galvatron-BMW always outperforms all baselines in multiple regards. For instance, on ViT, Galvatron-BMW promotes the overall system throughput by up to 493% compared with pure parallelism strategies, and achieves a maximum of 173% acceleration compared with hybrid strategies, including automatic parallelism with limited dimensions and expert-designed 3D parallelism. Similarly, on the other three models, Galvatron-BMW achieves a maximum of 407%-530% and 150%-242% compared with single and hybrid strategies respectively. It is worthy to note that even for models that can fit into GPU memory, TP, PP, or SDP, which incur additional communication overheads compared to DP, may still outperform DP in terms of overall system throughput. This phenomenon is not surprising for two reasons. First, DP has to replicate the entire model, limiting the achievable batch size, which may lead to under-utilization of GPUs. Second, even with the same batch size, the communication overheads in model parallelism mainly come from activations, which can sometimes be smaller than

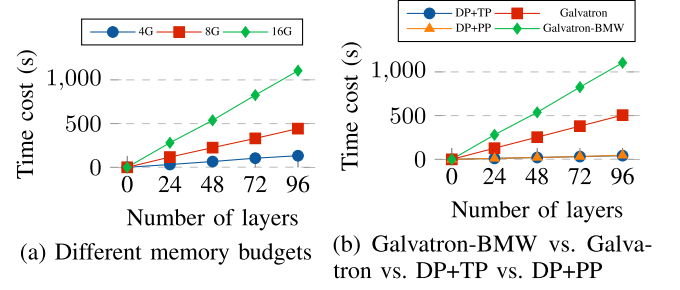


Fig. 5. Search time costs with different numbers of layers.

gradients in DP. By balancing multiple parallelism dimensions, Galvatron-BMW achieves an optimal trade-off between memory, computation, and communication costs, resulting in larger batch sizes and higher throughput.

Then, we look carefully into the effectiveness of each technique in Galvatron-BMW. In comparison to Galvatron, we observe that by integrating CKPT into search space, Galvatron-Base amplifies the overall system throughput by up to an impressive 109%. Such enhancement is attributed to CKPT's memory efficiency, which facilitates Galvatron-Base to achieve larger training batch size (e.g., a batch size of 160 for BERT-Huge-32 under 20 GB memory constraint), thereby optimizing throughput. Furthermore, Galvatron (1F1B+Bi-obj), which enables bi-objective optimization and strikes a balance between memory and computational workload, bolsters the system throughput by up to 44% compared to Galvatron. Benefiting from these two techniques, Galvatron-BMW manages to achieve state-of-the-art performance across all models.

We can also find that different models may have different preferences on the parallelism strategies. For example, under different memory budgets, BERT almost always prefers DP+PP among all baselines. Similar observations could be also found on some cases of T5. For ViT and Swin, the preferences change to SDP when increasing the memory budgets. The reason mainly comes from that NLP models have larger activation while CV models have larger model parameters, thus the latter could benefit more from sharding the model parameters across the GPUs. Here DeepSpeed 3D uses an officially suggested strategy [54] combining 2-way DP/TP/PP together. Such a fixed strategy outperforms three pure parallelisms but fails to beat SDP in most cases. Another interesting finding is that the hybrid parallelisms like DP+TP and DP+PP may perform worse than pure SDP (e.g., ViT-Huge-32 with 8 G, Swin-Huge-32 with 16 G). It further indicates that existing automatic parallelism approaches focusing on limited model parallelism dimensions are suffering from these limitations.

C. Optimization Efficiency

The efficiency of our dynamic programming search algorithm varies according to different number of model layers, overall strategies and memory constraints. As shown in Fig. 5(a), when the number of model layers and memory limit increase linearly, the search time of our algorithm increases linearly as expected,

TABLE II
COMPARISON WITH 8 GPUS UNDER DIFFERENT MEMORY CONSTRAINTS

Memory	Strategy	BERT-Huge-32	BERT-Huge-48	ViT-Huge-32	ViT-Huge-48	T5-Large-32	T5-Large-48	Swin-Huge-32	Swin-Huge-48
8G	PyTorch DDP (DP)	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM
	Megatron (TP)	OOM	OOM	16.16 (24)	10.65 (16)	OOM	OOM	13.47 (24)	8.41 (8)
	PyTorch GPipe (PP)	OOM	OOM	20.57 (56)	16.59 (32)	OOM	OOM	23.61 (40)	16.42 (24)
	FSDP/ZeRO-3 (SDP)	4.65 (8)	OOM	33.25 (64)	15.71 (40)	5.97 (8)	OOM	24.86 (48)	11.92 (32)
	DeepSpeed 3D	7.79 (8)	OOM	30.56 (40)	14.59 (16)	8.12 (8)	OOM	26.22 (32)	14.27 (16)
	Galvatron (DP+TP)	OOM	OOM	29.4 (32)	15.76 (16)	OOM	OOM	26.18 (24)	14.76 (16)
	Galvatron (DP+PP)	OOM	OOM	31.79 (48)	20.93 (24)	9.37 (8)	OOM	27.18 (40)	17.71 (24)
	Galvatron	8.16 (8)	OOM	36.58 (56)	20.93 (24)	9.37 (8)	OOM	31.33 (48)	21.64 (32)
	Galvatron-Base	17.04 (88)	9.99 (72)	64.03 (544)	35.89 (512)	11.50 (56)	6.87 (56)	34.55 (328)	25.69 (292)
	Galvatron (1F1B+Bi-obj)	10.05 (8)	OOM	41.85 (56)	23.10 (40)	9.52 (16)	OOM	32.95 (56)	22.25 (48)
	Galvatron-BMW	19.06 (184)	10.23 (176)	71.57 (568)	38.75 (544)	20.74 (80)	12.74 (64)	65.34 (360)	34.21 (360)
12G	PyTorch DDP (DP)	OOM	OOM	14.22 (16)	OOM	OOM	OOM	OOM	OOM
	Megatron (TP)	5.72 (8)	OOM	16.71 (48)	10.99 (32)	5.14 (8)	OOM	13.68 (40)	9.62 (24)
	PyTorch GPipe (PP)	9.22 (8)	6.2 (8)	25.13 (104)	16.62 (64)	9.09 (8)	6.83 (8)	26.07 (72)	19.82 (48)
	FSDP/ZeRO-3 (SDP)	8.91 (16)	3.15 (8)	47.41 (112)	24.24 (72)	11.26 (16)	4.11 (8)	37.38 (88)	21.98 (64)
	DeepSpeed 3D	7.79 (8)	5.35 (8)	37.88 (80)	22.68 (48)	8.12 (8)	5.76 (8)	34.14 (72)	20.07 (40)
	Galvatron (DP+TP)	8.92 (8)	5.35 (8)	42.21 (64)	17.2 (32)	9.53 (8)	OOM	37.26 (56)	20.18 (32)
	Galvatron (DP+PP)	9.22 (8)	6.2 (8)	50.69 (72)	24.01 (56)	11.95 (16)	6.83 (8)	35.87 (56)	21.69 (48)
	Galvatron	11.39 (16)	6.2 (8)	50.69 (72)	26.63 (72)	14.49 (16)	6.83 (8)	41.69 (64)	25.42 (64)
	Galvatron-Base	18.66 (72)	12.14 (136)	68.46 (568)	43.78 (536)	20.66 (80)	11.14 (72)	58.56 (360)	30.70 (328)
	Galvatron (1F1B+Bi-obj)	15.20 (32)	6.82 (8)	54.92 (104)	31.79 (72)	15.24 (24)	9.06 (8)	57.73 (136)	27.02 (96)
	Galvatron-BMW	24.37 (192)	15.97 (192)	79.24 (512)	46.94 (568)	24.07 (96)	15.13 (80)	76.86 (384)	47.78 (376)
16G	PyTorch DDP (DP)	6.39 (8)	OOM	44.40 (64)	OOM	7.79 (8)	OOM	28.61 (40)	OOM
	Megatron (TP)	6.06 (16)	3.88 (8)	16.81 (72)	11.02 (40)	5.14 (8)	OOM	13.83 (56)	9.71 (40)
	PyTorch GPipe (PP)	12.96 (16)	6.2 (8)	25.26 (144)	17.24 (96)	9.09 (8)	6.83 (8)	28.23 (104)	20.11 (64)
	FSDP/ZeRO-3 (SDP)	12.47 (24)	6.06 (16)	59.93 (160)	32.15 (104)	14.95 (24)	7.16 (16)	49.68 (136)	26.46 (88)
	DeepSpeed 3D	8.50 (16)	5.35 (8)	41.67 (128)	25.45 (72)	11.52 (16)	5.76 (8)	37.13 (104)	24.12 (64)
	Galvatron (DP+TP)	12.59 (16)	6.19 (8)	46.02 (88)	23.97 (48)	14.52 (16)	6.84 (8)	44.65 (80)	26.51 (48)
	Galvatron (DP+PP)	13.00 (16)	6.2 (8)	54.05 (120)	28.01 (56)	14.64 (16)	6.83 (8)	44.15 (96)	25.82 (56)
	Galvatron	15.05 (24)	7.46 (16)	63.25 (160)	35.74 (104)	16.50 (24)	8.36 (16)	54.06 (136)	29.21 (72)
	Galvatron-Base	21.70 (128)	12.92 (96)	75.98 (744)	45.75 (656)	23.67 (88)	15.69 (80)	68.98 (264)	41.15 (360)
	Galvatron (1F1B+Bi-obj)	16.61 (32)	10.03 (32)	88.86 (480)	39.39 (104)	16.72 (24)	9.69 (16)	73.20 (392)	39.34 (136)
	Galvatron-BMW	27.90 (224)	17.37 (208)	89.22 (520)	50.34 (480)	26.75 (128)	17.05 (112)	84.52 (428)	52.78 (392)
20G	PyTorch DDP (DP)	11.57 (16)	OOM	61.54 (112)	17.02 (32)	14.3 (16)	5.43 (8)	42.82 (80)	11.8 (24)
	Megatron (TP)	6.06 (16)	3.88 (8)	16.11 (88)	11.02 (56)	5.47 (16)	3.55 (8)	13.84 (72)	9.79 (48)
	PyTorch GPipe (PP)	13.52 (24)	7.05 (16)	28.64 (192)	17.96 (128)	9.53 (16)	8.13 (16)	29.75 (128)	20.73 (88)
	FSDP/ZeRO-3 (SDP)	17.06 (40)	7.8 (24)	63.75 (216)	38.29 (136)	17.93 (32)	7.16 (16)	55.22 (176)	32.63 (120)
	DeepSpeed 3D	8.50 (16)	5.35 (8)	43.36 (168)	27.82 (104)	13.14 (24)	7.96 (16)	40.60 (136)	26.09 (96)
	Galvatron (DP+TP)	14.65 (24)	8.05 (16)	61.54 (112)	28.69 (72)	15.35 (24)	6.84 (8)	54.87 (104)	30.59 (72)
	Galvatron (DP+PP)	15.52 (24)	8.11 (16)	61.54 (112)	34.88 (96)	17.27 (24)	10.33 (16)	50.19 (136)	31.62 (80)
	Galvatron	18.21 (40)	8.95 (24)	70.5 (152)	41.2 (136)	18.64 (32)	10.33 (16)	60.06 (144)	37.75 (120)
	Galvatron-Base	22.57 (160)	14.90 (136)	77.96 (1232)	52.06 (800)	28.06 (96)	16.33 (96)	73.26 (392)	47.43 (520)
	Galvatron (1F1B+Bi-obj)	22.63 (64)	11.16 (32)	91.65 (568)	51.07 (480)	26.81 (64)	11.45 (24)	76.10 (436)	51.20 (360)
	Galvatron-BMW	29.08 (256)	18.30 (232)	95.59 (592)	52.06 (800)	29.28 (152)	18.09 (128)	87.23 (512)	57.23 (456)

The maximum throughput (samples/s) of each strategy is given, along with the corresponding batch size in the bracket, and OOM denotes Out-Of-Memory.

The bold values represent the strategy with the highest system throughput and superior performance.

only hundreds of seconds are required to generate the optimal execution plan, which is acceptable and negligible relative to the extremely long model training time. Fig. 5(b) demonstrates the impact of total parallelism dimensions on the search time, both DP+TP and DP+PP have a total of 4 alternate strategies on 8 GPUs, while Galvatron-BMW and Galvatron has 44 and 22 overall candidates. In this case, the search time of DP+TP and DP+PP is consistent and much less than that of Galvatron and Galvatron-BMW.

D. Scalability Study

We conduct further comparisons on large clusters. We first extend our experiments to 16 Nvidia RTX TITAN GPUs over two servers connected by 100 Gb InfiniBand network (referred to as low-performance cluster), as well as 16 Nvidia A100 GPUs with NVLink over two servers connected by 100 Gb InfiniBand network (referred to as high-performance cluster). Table III illustrates the results on BERT, ViT and T5-512/4 models. Not surprisingly, Galvatron-BMW achieves the best performance with different memory budgets on both two clusters. On low-performance cluster, compared with the results on 8 GPUs, Galvatron-BMW and the hybrid parallelism methods could obtain more than $2\times$ speedups for many cases.

For example, Galvatron-BMW enlarges the batch size from 520 to 960 for ViT-Huge-32 under 16 GPUs with 16 GB memory, and the throughput increases from 89.22 to 179.89 samples per second. The $2.02\times$ speedup comes from the flexible fine-grained layer-level parallelism strategy, which helps to reduce the communication costs and improve the training efficiency. As can be seen, Galvatron-BMW outperforms all baselines across both two clusters as well as different models (including imbalanced model T5-512/4). This highlights Galvatron-BMW's adaptive capability when handling different clusters and diverse model workloads. We then extend to an larger GPU cluster including 64 Nvidia A100 40 GB GPUs, where each server has 8 GPUs equipped with NVLink and the servers are connected by 100 Gb InfiniBand network. Since the environment scale is significantly larger than before, we also increase the model sizes to 10 billion parameters (i.e., BERT-xHuge and ViT-xHuge, details are in Table I). As we can see in Table IV, even on such a large GPUs cluster, Galvatron-BMW still outperforms these baseline methods. Besides, based on our observations, the search time costs do not exponentially grow (i.e., $2.2\times$ and $9.2\times$ for 16 GPUs and 64 GPUs respectively compared with 8 GPUs), which is still tolerable.

We further verify the scalability to model size by experimenting on larger LLMs, GPT-3 15B, 39B and 65B. Given

TABLE III
COMPARISON WITH 16 GPUS

Memory	Strategy	BERT-Huge-32	BERT-Huge-48	ViT-Huge-32	ViT-Huge-48	T5-512/4-32	T5-512/4-48
8G Low-perf Cluster	PyTorch DDP (DP)	OOM	OOM	OOM	OOM	OOM	OOM
	Megatron (TP)	OOM	OOM	16.86 (32)	10.86 (16)	11.38 (16)	OOM
	PyTorch GPipe (PP)	13.79 (16)	5.88 (8)	50.70 (128)	27.96 (80)	21.45 (32)	14.95 (32)
	FSDP/ZERO-3 (SDP)	8.95 (16)	6.12 (16)	69.48 (128)	34.92 (96)	31.63 (48)	15.94 (32)
	DeepSpeed 3D	15.24 (16)	6.43 (8)	57.14 (64)	29.92 (40)	27.97 (32)	14.03 (16)
	Galvatron (DP+TP)	OOM	OOM	54.43 (64)	24.56 (32)	22.37 (16)	15.21 (16)
	Galvatron (DP+PP)	13.91 (16)	5.88 (8)	68.56 (128)	35.02 (72)	33.63 (40)	18.99 (32)
	Galvatron	15.24 (16)	8.43 (16)	76.74 (128)	38.32 (88)	36.14 (48)	21.25 (32)
	Galvatron (1F1B+Bi-obj)	25.02 (88)	12.00 (64)	120.90 (544)	66.77 (384)	54.71 (72)	23.07 (48)
	Galvatron-Base	38.41 (160)	15.06 (128)	111.21 (672)	62.84 (648)	63.84 (192)	34.38 (208)
	Galvatron-BMW	42.04 (256)	20.36 (224)	139.51 (928)	78.32 (784)	94.03 (256)	56.49 (256)
16G Low-perf Cluster	PyTorch DDP (DP)	12.14 (16)	OOM	88.06 (128)	OOM	43.05 (48)	10.92 (16)
	Megatron (TP)	6.12 (16)	4.23 (16)	17.11 (64)	11.26 (48)	12.53 (32)	8.00 (16)
	PyTorch GPipe (PP)	23.29 (40)	12.92 (24)	69.72 (320)	50.23 (208)	36.09 (80)	26.11 (80)
	FSDP/ZERO-3 (SDP)	30.37 (64)	11.74 (32)	123.95 (320)	61.49 (224)	68.92 (112)	36.33 (80)
	DeepSpeed 3D	23.92 (48)	13.03 (24)	91.56 (256)	53.81 (152)	49.23 (88)	28.45 (56)
	Galvatron (DP+TP)	23.01 (32)	10.50 (16)	99.22 (160)	49.82 (96)	73.82 (80)	34.71 (48)
	Galvatron (DP+PP)	23.73 (40)	13.12 (40)	115.88 (224)	61.38 (208)	69.42 (96)	32.76 (56)
	Galvatron	32.67 (64)	14.74 (40)	131.15 (320)	72.74 (208)	73.82 (80)	40.10 (64)
	Galvatron (1F1B+Bi-obj)	49.65 (232)	22.30 (104)	167.48 (832)	100.26 (512)	92.90 (320)	68.94 (288)
	Galvatron-Base	48.66 (368)	25.11 (200)	163.92 (1376)	91.85 (904)	87.58 (384)	55.12 (288)
	Galvatron-BMW	55.57 (384)	33.14 (320)	179.89 (960)	114.08 (1136)	117.71 (400)	80.53 (488)
8G High-perf Cluster	PyTorch DDP (DP)	OOM	OOM	OOM	OOM	OOM	OOM
	Megatron (TP)	OOM	OOM	53.43 (32)	31.92 (16)	30.75 (16)	OOM
	PyTorch GPipe (PP)	57.22 (16)	14.74 (8)	233.45 (128)	176.60 (80)	84.23 (32)	55.81 (32)
	FSDP/ZERO-3 (SDP)	27.75 (16)	17.80 (16)	204.73 (128)	119.91 (96)	107.13 (48)	47.11 (32)
	DeepSpeed 3D	85.38 (16)	42.39 (8)	292.64 (64)	185.23 (40)	124.71 (32)	70.36 (16)
	Galvatron (DP+TP)	OOM	OOM	126.05 (48)	38.65 (32)	48.02 (16)	32.69 (16)
	Galvatron (DP+PP)	74.80 (16)	35.42 (8)	412.70 (104)	244.97 (80)	142.34 (40)	75.05 (32)
	Galvatron	99.17 (24)	53.38 (16)	453.87 (152)	277.94 (96)	152.71 (48)	83.86 (32)
	Galvatron (1F1B+Bi-obj)	162.52 (96)	81.04 (64)	572.05 (576)	407.64 (384)	216.47 (128)	93.53 (48)
	Galvatron-Base	140.21 (144)	94.33 (136)	576.94 (832)	369.09 (704)	227.40 (288)	151.96 (248)
	Galvatron-BMW	202.34 (296)	132.03 (288)	697.34 (1152)	457.74 (1056)	348.29 (416)	230.74 (384)
16G High-perf Cluster	PyTorch DDP (DP)	34.95 (16)	OOM	303.26 (128)	OOM	126.02 (48)	28.15 (16)
	Megatron (TP)	18.87 (16)	12.55 (16)	55.68 (64)	35.05 (48)	35.18 (32)	21.97 (16)
	PyTorch GPipe (PP)	91.10 (40)	27.42 (24)	323.95 (320)	254.12 (208)	154.53 (80)	98.55 (80)
	FSDP/ZERO-3 (SDP)	113.98 (64)	38.89 (32)	514.80 (320)	274.47 (224)	212.35 (112)	107.24 (80)
	DeepSpeed 3D	133.67 (48)	82.25 (32)	506.17 (256)	322.73 (152)	188.74 (88)	99.94 (56)
	Galvatron (DP+TP)	72.37 (32)	25.65 (16)	482.42 (208)	165.92 (96)	200.28 (80)	95.48 (48)
	Galvatron (DP+PP)	135.48 (48)	74.95 (32)	611.15 (248)	346.13 (128)	230.16 (96)	115.55 (56)
	Galvatron	147.26 (64)	95.90 (40)	641.40 (352)	398.28 (232)	238.56 (96)	127.34 (72)
	Galvatron (1F1B+Bi-obj)	226.73 (192)	144.59 (144)	797.16 (920)	488.44 (896)	349.72 (192)	207.08 (192)
	Galvatron-Base	168.08 (376)	113.63 (344)	727.98 (1344)	440.82 (1152)	323.79 (440)	213.65 (400)
	Galvatron-BMW	243.55 (416)	161.07 (320)	871.26 (1344)	533.46 (1088)	409.47 (512)	265.37 (480)

The bold values represent the strategy with the highest system throughput and superior performance.

TABLE IV
COMPARISON WITH 64 GPUS

Memory	Strategy	BERT-xHuge	ViT-xHuge
16G High-perf Cluster	PyTorch DDP (DP)	OOM	OOM
	Megatron (TP)	0.68 (3)	1.94 (12)
	PyTorch GPipe (PP)	9.74 (16)	61.95 (96)
	FSDP/ZERO-3 (SDP)	OOM	OOM
	DeepSpeed 3D	8.44 (16)	64.91 (96)
	Galvatron (DP+TP)	1.73 (4)	5.07 (2)
	Galvatron (DP+PP)	9.74 (16)	64.83 (104)
	Galvatron	13.77 (24)	68.35 (136)
	Galvatron (1F1B+Bi-obj)	19.43 (24)	109.81 (256)
	Galvatron-Base	25.30 (128)	130.88 (768)
	Galvatron-BMW	37.09 (376)	209.48 (1392)
32G High-perf Cluster	PyTorch DDP (DP)	OOM	OOM
	Megatron (TP)	0.77 (7)	2.11 (28)
	PyTorch GPipe (PP)	21.38 (48)	94.84 (288)
	FSDP/ZERO-3 (SDP)	OOM	OOM
	DeepSpeed 3D	21.28 (40)	91.19 (256)
	Galvatron (DP+TP)	1.73 (4)	5.51 (68)
	Galvatron (DP+PP)	23.64 (48)	110.98 (232)
	Galvatron	27.49 (64)	114.55 (328)
	Galvatron (1F1B+Bi-obj)	41.11 (80)	180.08 (640)
	Galvatron-Base	40.17 (184)	195.40 (1024)
	Galvatron-BMW	60.04 (640)	330.30 (1744)

The bold values represent the strategy with the highest system throughput and superior performance.

the substantial size of these LLMs, we compare on GPUs with larger memory capacity and higher communication bandwidth (32 Nvidia A100 80 GB GPUs connected by 400 Gb InfiniBand).

We also compare with Alpa [51] which automates parallelism on JAX. As shown in Table VI, Galvatron-BMW's performance enhancement is more pronounced with larger models. Alpa, despite its broader operator-level search space, primarily treats SDP as a secondary optimization, allowing only DP or SDP, but not both, across the entire model, thus missing a more nuanced DP-SDP trade-off. Furthermore, CKPT proves a critical dimension for larger models under resource constraints, which is not considered in Alpa's search space. In contrast, Galvatron-BMW effectively balances memory, computation, and communication, delivering superior performance consistently.

E. Ablation Study

To delve deeper into the performance of bi-objective optimization of workload balance, we conduct experiments on different pipeline partitions. The results of BERT and T5-512/4 on high-performance cluster (16 A100) are shown in Table V. The memory-balanced partition can evidently support the training of larger batch sizes for the provided models, enabling a batch size of 128 for T5-512/4-32 under 8 GB memory, in contrast to the limit of 48 in time-balanced partition. On the other hand, the time-balanced partition encourages a more uniform workload distribution across varying pipeline stages. As an example, a balanced partition of [16,16] is accomplished for the BERT-Huge-32, which has homogeneous layers. Indeed, bi-objective

TABLE V
ABLATION STUDY ON BI-OBJECTIVE OPTIMIZATION OF WORKLOAD BALANCE ON HIGH-PERFORMANCE CLUSTER (16 A100)

Memory	Strategy	BERT-Huge-32		BERT-Huge-48		T5-512/4-32		T5-512/4-48	
8G High-perf Cluster	Galvatron (1F1B+Mem)	139.27 (96)	[11,21]	74.13 (64)	[6,9,13,20]	198.23 (128)	[6,26]	87.19 (48)	[9,39]
	Galvatron (1F1B+Time)	104.17 (24)	[16,16]	53.38 (16)	[24,24]	161.52 (48)	[14,18]	82.45 (24)	[10,10,13,15]
	Galvatron (1F1B+Bi-obj)	162.52 (96)	[14,18]	81.04 (64)	[7,9,13,19]	216.47 (128)	[8,24]	93.53 (48)	[12,36]
16G High-perf Cluster	Galvatron (1F1B+Mem)	201.54 (376)	[10,22]	121.24 (192)	[16,32]	316.03 (288)	[6,26]	184.89 (224)	[9,39]
	Galvatron (1F1B+Time)	226.73 (192)	[16,16]	134.72 (96)	[24,24]	349.72 (192)	[11,21]	191.88 (128)	[17,31]
	Galvatron (1F1B+Bi-obj)	226.73 (192)	[16,16]	144.59 (144)	[22,26]	349.72 (192)	[11,21]	207.08 (192)	[11,37]

The performance of bi-objective optimization (1F1B+Bi-obj) is compared with that of memory-balanced partition (1F1B+Mem) and time-balanced partition (1F1B+Time). The pipeline partition p is shown next to the system throughput and training batch size.

The bold values represent the strategy with the highest system throughput and superior performance.

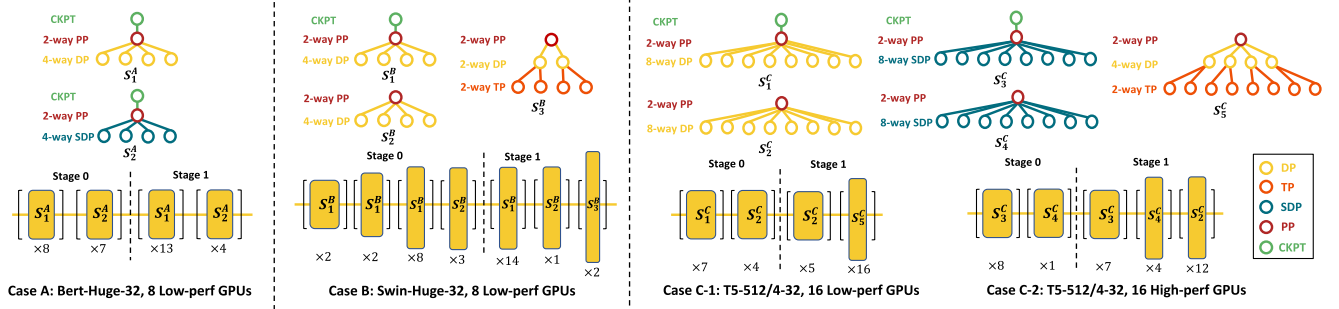


Fig. 6. Examples of the optimal parallelism plans given by Galvatron-BMW. Case A and B is for BERT-Huge-32 and Swin-Huge-32 under 8 GB memory budgets on 8 low-performance GPUs, and case C is for T5-512/4-32 under 8 GB memory budgets on 16 low-performance GPUs and 16 high-performance GPUs. Each yellow rectangle denotes an encoder layer, and its height and width represent parameter size and activation size respectively. The number $\times N$ under the rectangle means applying a strategy for consecutive N layers.

TABLE VI
COMPARISON OF LARGER LLMs ON 80 GB GPUs

Strategy	GPT3-15B	GPT3-39B	GPT3-65B
PyTorch DDP (DP)	OOM	OOM	OOM
Megatron (TP)	12.31 (24)	5.56 (20)	2.02 (2)
PyTorch GPipe (PP)	26.56 (128)	12.38 (64)	4.62 (24)
FSDP/ZeRO-3 (SDP)	45.88 (128)	14.31 (96)	3.40 (32)
DeepSpeed 3D	37.65 (128)	13.79 (48)	4.91 (16)
Galvatron (DP+TP)	14.36 (28)	6.82 (24)	2.99 (4)
Galvatron (DP+PP)	27.17 (72)	12.97 (48)	4.62 (24)
Galvatron	50.79 (128)	14.93 (96)	5.14 (16)
Galvatron (1F1B+Bi-obj)	56.57 (1280)	21.69 (768)	10.62 (512)
Galvatron-Base	57.33 (512)	21.14 (160)	13.97 (288)
Alpa	45.51 (1024)	13.75 (512)	7.16 (512)
Galvatron-BMW	61.42 (2560)	22.86 (2048)	13.97 (288)

The bold values represent the strategy with the highest system throughput and superior performance.

optimization harnesses the advantages of both memory- and time-balanced partition, striking an optimal balance between memory and time cost to yield superior performance. In the training of BERT-Huge-48 and T5-512/4-48 under a memory constraint of 16 G, the pipeline partitions are tuned to intermediary configurations of [22, 26] and [11, 37], while the training batch sizes are optimized as 144 and 192. These strategic adaptations culminate in marked improvements in system throughput, with surges of up to 19% and 12%, thus underscoring the effectiveness of bi-objective optimization.

F. Optimal Parallelism Plan

We list some examples of the optimal parallelism plans suggested by Galvatron-BMW in Fig. 6. For comparison, we choose different models (BERT-Huge-32, Swin-Huge-32,

and T5-512/4-32), different GPU numbers (8 and 16), and different GPU clusters. In case A for BERT, Galvatron-BMW provides an optimal strategy containing S_1^A , PP+DP+CKPT, and S_2^A , PP+SDP+CKPT, incorporating SDP and CKPT to reduce memory costs and enlarge the batch size as well as the throughput. In case B for Swin-Huge-32, the optimal plans given by Galvatron-BMW is rather complex, as it has four different layers which have different strategy preference. In Swin Transformer, shallower layers have larger activation size and smaller parameter size. To reduce memory consumption and communication overhead, shallower layers prefer data parallel which splits input activations and communicates parameter gradients, while deeper layers prefer tensor parallel which splits model parameters and communicates activations. Therefore, by mixing PP+DP, PP+DP+CKPT and PP+TP+DP, Galvatron-BMW maximizes the memory utilization as well as the system efficiency. To further analyze Galvatron-BMW, in case C, we test T5-512/4-32 on 16 low-performance GPUs and 16 high-performance GPUs under the same memory budget. In T5-512/4, as the decoder has shorter sequence length than the encoder, the activation size of the decoder is much less than the encoder, while the parameter size of the decoder is larger than the encoder due to the extra attention block. Therefore, similar to Swin, on low-performance GPUs, Galvatron-BMW prefers TP for the decoder to reduce communication volume, where S_5^C , PP+DP+TP, is recommended. However, on high-performance GPUs, the advantage of low communication volume is not significant due to the high communication bandwidth, and Galvatron-BMW mixes PP+SDP+CKPT, PP+SDP and PP+DP, preferring using SDP to reduce the memory consumption.

VIII. CONCLUSION

In this work, we presented Galvatron-BMW, a novel automatic parallel Transformer training system over multiple GPUs. Through the carefully designed search space decomposition and exploration algorithm, Galvatron-BMW significantly outperforms the state-of-the-art baselines on the training throughput. We hope the open source release of Galvatron-BMW will facilitate the future research directions on more challenging scenarios, e.g., heterogeneous environments and large DL models with complex and dynamic structures.

REFERENCES

- [1] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [2] H.-P. Wei et al., "A comparative study of CNN-and transformer-based visual style transfer," *J. Comput. Sci. Technol.*, vol. 37, no. 3, pp. 601–614, 2022.
- [3] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [4] Y. Xu, H. Cao, W. Du, and W. Wang, "A survey of cross-lingual sentiment analysis: Methodologies, models and evaluations," *Data Sci. Eng.*, vol. 7, no. 3, pp. 279–299, 2022.
- [5] D. Li, Y. Tu, X. Zhou, Y. Zhang, and Z. Ma, "End-to-end chinese entity recognition based on BERT-BiLSTM-att-CRF," *ZTE Commun.*, vol. 20, no. S1, pp. 27–35, 2022.
- [6] L. Deng and Y. Zhao, "Deep learning-based semantic feature extraction: A literature review and future directions," *ZTE Commun.*, vol. 21, no. 2, pp. 11–17, 2023.
- [7] T. B. Brown et al., "Language models are few-shot learners," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 159.
- [8] C. Ying et al., "Do transformers really perform bad for graph representation?," 2021, *arXiv:2106.05234*.
- [9] L. Li et al., "Memory-enhanced transformer for representation learning on temporal heterogeneous graphs," *Data Sci. Eng.*, vol. 8, no. 2, pp. 98–111, 2023.
- [10] F. Sun et al., "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 1441–1450.
- [11] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Annu. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
- [12] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [13] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, 2020, Art. no. 140.
- [14] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 9992–10002.
- [15] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. 38th Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 8748–8763.
- [16] A. Ramesh et al., "Zero-shot text-to-image generation," in *Proc. 38th Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 8821–8831.
- [17] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [18] M. Zinkevich, M. Weimer, A. Smola, and L. Li, "Parallelized stochastic gradient descent," in *Proc. 23rd Int. Conf. Neural Inf. Process. Syst.*, 2010, pp. 2595–2603.
- [19] J. Dean et al., "Large scale distributed deep networks," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1223–1231.
- [20] X. Miao et al., "Heterogeneity-aware distributed machine learning training via partial reduce," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 2262–2270.
- [21] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using megatron-LM," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, Art. no. 58.
- [22] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, Art. no. 10.
- [23] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, "Memory-efficient pipeline-parallel DNN training," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 7937–7947.
- [24] D. Narayanan et al., "PipeDream: Generalized pipeline parallelism for DNN training," in *Proc. 27th ACM Symp. Operating Syst. Princ.*, 2019, pp. 1–15.
- [25] B. Yang, J. Zhang, J. Li, C. Ré, C. R. Aberger, and C. D. Sa, "Pipemare: Asynchronous pipeline parallel DNN training," in *Proc. Mach. Learn. Syst.*, 2021, pp. 269–296.
- [26] Z. Jia, S. Lin, C. R. Qi, and A. Aiken, "Exploring hidden dimensions in parallelizing convolutional neural networks," in *Proc. 35th Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 2279–2288.
- [27] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," in *Proc. Mach. Learn. Syst.*, 2019, pp. 1–13.
- [28] C. Unger et al., "Unity: Accelerating dnn training through joint optimization of algebraic transformations and parallelization," in *Proc. 16th USENIX Symp. Operating Syst. Des. Implementation*, 2022, pp. 267–284.
- [29] M. Wang, C. Huang, and J. Li, "Supporting very large models using automatic dataflow graph partitioning," in *Proc. 14th EuroSys Conf.*, 2019, Art. no. 26.
- [30] Z. Cai et al., "TensorOpt: Exploring the tradeoffs in distributed DNN training with auto-parallelism," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1967–1981, Aug. 2022.
- [31] S. Fan et al., "DAPPLE: A pipelined data parallel approach for training large models," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2021, pp. 431–445.
- [32] S. Rajbhandari et al., "ZeRO: Memory optimizations toward training trillion parameter models," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, Art. no. 20.
- [33] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3505–3506.
- [34] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [35] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive language models beyond a fixed-length context," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 2978–2988.
- [36] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 7871–7880.
- [37] M. Baines et al., "FairScale: A general purpose modular PyTorch library for high performance and large scale training," 2021.
- [38] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, "Memory-efficient pipeline-parallel DNN training," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 7937–7947.
- [39] S. Smith et al., "Using DeepSpeed and megatron to train megatron-turing nlG 530b, a large-scale generative language model," 2022, *arXiv:2201.11990*.
- [40] Nvidia collective communications library (NCCL), 2021. [Online]. Available: <https://developer.nvidia.com/nccl>
- [41] S. Rashidi et al., "Enabling compute-communication overlap in distributed deep learning training platforms," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Architecture*, 2021, pp. 540–553.
- [42] Galvatron, 2023. [Online]. Available: <https://github.com/PKU-DAIR/Hetu-Galvatron>
- [43] X. Miao, X. Nie, H. Zhang, T. Zhao, and B. Cui, "Hetu: A highly efficient automatic parallel distributed deep learning system," *Sci. China Inf. Sci.*, vol. 66, 2022, Art. no. 117101.
- [44] X. Miao et al., "HET: Scaling out huge embedding model training via cache-enabled distributed framework," in *Proc. VLDB Endowment*, vol. 15, no. 2, pp. 312–320, 2022.
- [45] X. Miao et al., "HET-GMP: A graph-based system approach to scaling large embedding model training," in *Proc. Int. Conf. Manage. Data*, 2022, pp. 470–480.
- [46] X. Nie, P. Zhao, X. Miao, T. Zhao, and B. Cui, "HetuMoE: An efficient trillion-scale mixture-of-expert distributed training system," 2022, *arXiv:2203.14685*.
- [47] X. Miao et al., "Galvatron: Efficient transformer training over multiple GPUs using automatic parallelism," in *Proc. VLDB Endowment*, vol. 16, no. 3, pp. 470–479, 2022.
- [48] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, Art. no. 721.

- [49] S. Li et al., "PyTorch distributed: Experiences on accelerating data parallel training," in *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.
- [50] Y. Xu et al., "GSPMD: General and scalable parallelization for ML computation graphs," 2021, *arXiv:2105.04663*.
- [51] L. Zheng et al., "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," in *Proc. 16th USENIX Symp. Operating Syst. Des. Implementation*, 2022, pp. 559–578.
- [52] PyTorch gpipe, 2021. [Online]. Available: <https://pytorch.org/docs/stable/pipeline.html>
- [53] Y. Xu, H. Lee, D. Chen, H. Choi, B. Hechtman, and S. Wang, "Automatic cross-replica sharding of weight update in data-parallel training," 2020, *arXiv:2004.13336*.
- [54] Deepspeed 3D parallelism, 2021. [Online]. Available: https://github.com/microsoft/Megatron-DeepSpeed/blob/main/examples/pretrain_bert_distributed_with_mp.sh



Yujie Wang is currently working toward the PhD degree in computer science with Peking University. His research interests include distributed deep learning algorithm and system, memory management and optimization, and parallel computing framework.



Youhe Jiang is a research assistant with Peking University. His research interests are gpu acceleration, distributed deep learning system and network topology optimization.



Xupeng Miao received the PhD degree from the School of Computer Science, Peking University, in 2022. He is a postdoctoral researcher with the Computer Science Department of Carnegie Mellon University. His research interests include machine learning systems, data management and distributed computing.



Fangcheng Fu received the PhD degree in computer science from Peking University, China, in 2023. He is currently a Boya postdoctoral researcher with the School of CS, Peking University. His research interests include large-scale machine learning algorithm and systems, federated learning, and privacy-enhancing computation.



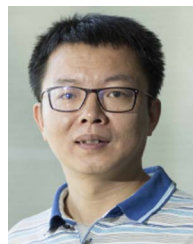
Shenhan Zhu is currently working toward the master's degree with the School of Software and Microelectronics, Peking University. His research interests include efficient Transformer and deep learning systems.



Xiaonan Nie is currently working toward the PhD degree in computer science with Peking University. His research interests include distributed deep learning system, heterogeneous memory management and sparse neural networks.



Yaofeng Tu received the PhD degree from the Nanjing University of Aeronautics and Astronautics, China, in 2019. He is currently a research professor in ZTE company. His main research interests include distributed computing, Big Data system, and machine learning.



Bin Cui (Fellow, IEEE) is a professor and vice dean with the School of CS, Peking University, China. He is a Cheung Kong distinguished professor, in 2016. His research interests include database systems, and data mining. He has published more than 200 research papers, and is the winner of Microsoft Young Professorship award (MSRA 2008), and CCF Young Scientist award (2009).